

Répartition des tâches — Équipe CyberCopilot (v2)

Projet : Assistant SOC intelligent basé sur LLM

Équipe : 4 personnes

Stratégie : chaque personne contribue **une fonctionnalité de code** clairement identifiable dans Git + sa partie documentaire / présentation

La logique de répartition

Le code de base est **déjà fonctionnel** (92 tests pytest, 6 types d'attaques, 4 formats de logs). Pour que chacun ait sa contribution code visible dans l'historique GitHub, on a découpé **4 mini-fonctionnalités indépendantes** que chaque personne peut implémenter sans casser l'existant.

Chaque mini-fonctionnalité est :

- **Indépendante** des autres (pas de conflit de merge)
- **Petite** (1 à 3 heures de code maximum)
- **Visible** dans l'application
- **Testable** en autonomie

Une fois codée, chaque personne fait son **propre commit Git** sous son nom, ce qui prouve sa contribution.

Personne 1 — Dashboard HTML interactif

Mission code (2 h)

Créer un **tableau de bord HTML statique** qui présente joliment tous les incidents détectés, avec graphiques et couleurs. C'est un fichier `.html` autonome qu'on peut ouvrir dans n'importe quel navigateur.

Fichiers à créer :

- `src/dashboard.py` — module de génération HTML
- Ajout d'une option dans le menu interactif (option 15)
- `tests/test_dashboard.py` — tests basiques

Ce que ça fait concrètement :

```
python app.py
# → choisir l'option 15 "Générer un dashboard HTML"
# → produit dashboard.html
# → ouvrir dashboard.html dans Firefox / Chrome
```

L'HTML doit contenir :

- En-tête avec logo CyberCopilot
- Compteurs : nombre d'incidents par sévérité (avec couleurs)
- Tableau de tous les incidents
- Style propre (CSS intégré, pas de framework externe)

Mission non-code

- Capture d'écran du dashboard pour les livrables
- Maquette Figma (voir brief du livrable 5)
- Logo CyberCopilot finalisé

Commit attendu

```
git commit -m "Add HTML dashboard generator with severity stats and incident table"
```

Personne 2 — Intégration continue GitHub Actions

Mission code (1 h 30)

Configurer un **workflow GitHub Actions** qui exécute automatiquement les 92 tests pytest à **chaque push** sur le repo. Si un test casse, GitHub affiche une croix rouge. Si tout passe, un beau badge vert apparaît dans le README.

Fichiers à créer :

- `.github/workflows/tests.yml` — workflow CI
- Mise à jour du `README.md` pour ajouter le badge de statut
- Documentation du processus dans `CONTRIBUTING.md`

Contenu de `tests.yml` à respecter :

```
name: Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt
      - run: pytest tests/ -v
```

Mission non-code

- Réviser et améliorer le `README.md`
- Ajouter une licence MIT
- Configurer les Topics GitHub (cybersecurity, soc, llm, python, siem)

Commit attendu

```
git commit -m "Add GitHub Actions CI workflow + status badge in README"
```

Bonus : ce commit ajoute un **vrai signal de qualité professionnelle** au projet (les recruteurs regardent ça).

Personne 3 — Système d'aide intégré + glossaire

Mission code (1 h 30)

Ajouter une **commande d'aide intelligente** dans l'application qui explique chaque type d'attaque, chaque commande, et les acronymes (SOC, SIEM, LLM, IoC...). Utile pour les analystes juniors et pédagogique pour la soutenance.

Fichiers à créer :

- `src/help.py` — contient les définitions et explications
- Ajout d'une option dans le menu (option 16 : " Aide et glossaire")
- Sous-menu interactif :
 1. Expliquer un type d'attaque
 2. Glossaire (SOC, SIEM, LLM...)

- 3. Commandes disponibles
- 4. FAQ

Contenu attendu de `help.py` :

```
ATTACK_EXPLANATIONS = {
    "brute_force_ssh": {
        "title": "Brute Force SSH",
        "description": "Un attaquant essaie des milliers...",
        "how_to_protect": "fail2ban, clés SSH, port différent",
        "real_world_example": "L'attaque sur GitHub en 2013...",
    },
    "sql_injection": { ... },
    # etc. pour les 6 types
}

GLOSSARY = {
    "SOC": "Security Operations Center – équipe...",
    "SIEM": "Security Information and Event Management...",
    "LLM": "Large Language Model...",
    # etc.
}
```

Mission non-code

- Compiler le rapport final fusionnant les 6 livrables
- Ajouter une page de garde au rapport
- Créer un sommaire général

Commit attendu

```
git commit -m "Add integrated help system with attack explanations and glossary"
```

Personne 4 — Mode démo automatique + capture vidéo

Mission code (2 h)

Créer un **script de démonstration** qui enchaîne automatiquement toutes les détections avec des pauses lisibles, parfait pour la soutenance ou un screencast. Plus besoin de cliquer 15 fois pendant la présentation.

Fichiers à créer :

- `demo.py` (à la racine du projet) — script de démo
- `src/demo_player.py` — utilitaires d'affichage avec pauses

Ce que ça fait concrètement :

```
python demo.py
# → titre animé "CyberCopilot – Démo soutenance B2"
# → "Étape 1/5 : Détection brute force SSH..."
# → analyse en direct, affichage de l'incident
# → pause de 5 secondes (lisible par le public)
# → "Étape 2/5 : Injection SQL..."
# → etc.
# → résumé final : "6 incidents détectés, X tokens économisés"
```

Spécifications :

- Affichage d'un titre stylé entre chaque étape
- Pauses configurables (`--speed slow/normal/fast`)
- Mode `--silent` pour enregistrement vidéo sans interactions
- Statistiques finales

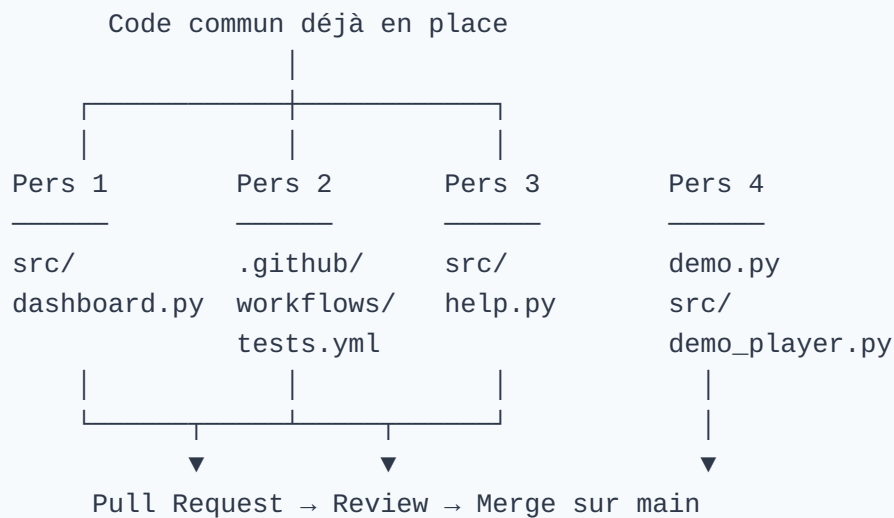
Mission non-code

- Préparer les slides PowerPoint de soutenance
- Enregistrer la vidéo de démo avec OBS Studio
- Rédiger le script du pitch oral

Commit attendu

```
git commit -m "Add automated demo script with paced execution for presentations"
```

Coordination entre les rôles



Chacun travaille sur **sa branche dédiée** :

- `feature/dashboard-html` (Personne 1)
- `feature/github-actions` (Personne 2)
- `feature/help-system` (Personne 3)
- `feature/demo-script` (Personne 4)

Puis Pull Request → Lead technique (toi) valide et merge sur `main` .

Workflow Git pour chaque personne

Étape 1 — Cloner le projet

```
git clone https://github.com/abdoul-latif-dev/cybercopilot.git
cd cybercopilot
```

Étape 2 — Créer sa branche perso

```
git checkout -b feature/dashboard-html # adapter selon le rôle
```

Étape 3 — Configurer son identité Git (UNE FOIS)

```
git config user.name "Ton Nom"
git config user.email "ton.email@ece.fr"
```

C'est **crucial** pour que ton nom apparaisse dans les commits.

Étape 4 — Coder, tester, commit

```
# Coder...
pytest tests/ # vérifier que rien n'est cassé
git add .
git commit -m "Add HTML dashboard generator"
```

Étape 5 — Pousser sur GitHub

```
git push -u origin feature/dashboard-html
```

Étape 6 — Créer une Pull Request

- Aller sur <https://github.com/abdoul-latif-dev/cybercopilot>
- Onglet "Pull requests" → "New pull request"
- Sélectionner sa branche
- Demander la review au lead technique



Règles à respecter par tout le monde

1. **Ne JAMAIS toucher** au code existant (parser, detector, llm_client, etc.)
 - Seules les nouvelles fonctionnalités sont autorisées
 - Si une modification s'impose, en parler au lead avant
 2. **Ne JAMAIS commiter** le fichier `.env` (clé API)
 - Il est déjà dans `.gitignore`, vérifier qu'il y reste
 3. **Toujours lancer** `pytest` avant de commit
 - Si un test casse → corriger avant le push
 4. **Commits clairs** en français ou anglais, mais cohérent
 5. **Une fonctionnalité = une Pull Request**
 - Pas de PR géante mélangeant 3 sujets
-

Timeline suggérée (sur 5 jours)

Jour	Personne 1	Personne 2	Personne 3	Personne 4
J1	Clone + setup	Clone + setup	Clone + setup	Clone + setup
J2	Dashboard HTML	GitHub Actions	Help system	Demo script
J3	Tests + commit	Tests + commit	Tests + commit	Tests + commit
J4	PR + review	PR + review	PR + review	PR + review
J5	Merge + visuels	Merge + README	Merge + rapport	Merge + slides

Visibilité pour le prof

Après ces contributions, l'historique Git affichera :

```
$ git log --oneline --author="Personne 1"
abc1234 Add HTML dashboard generator with severity stats and incident table

$ git log --oneline --author="Personne 2"
def5678 Add GitHub Actions CI workflow + status badge in README

$ git log --oneline --author="Personne 3"
ghi9012 Add integrated help system with attack explanations and glossary

$ git log --oneline --author="Personne 4"
jkl3456 Add automated demo script with paced execution for presentations
```

Le prof, en regardant l'historique, verra **clairement** que les 4 personnes ont contribué techniquement, chacune avec sa pierre.

Pourquoi cette répartition est intelligente

- **Aucun risque** pour le code existant (chacun travaille sur des fichiers nouveaux)
- **Chacun a sa "vitrine"** : une fonctionnalité concrète qu'il peut montrer
- **L'historique Git est propre** : 4 contributions claires, pas de mélange
- **Les fonctionnalités sont utiles** (pas du code "bidon" juste pour faire des commits)
- **Tout reste dans le scope** du cahier des charges



Si quelqu'un n'arrive pas à faire sa partie

Le lead technique (Abdoul-Latif) peut aider, mais l'objectif est que **chacun fasse au moins le minimum lui-même**. Pour rappel, ce sont des fonctionnalités vraiment simples :

- Personne 1 : c'est un fichier HTML + un peu de Python pour le générer
- Personne 2 : c'est un fichier YAML de 15 lignes
- Personne 3 : c'est un dictionnaire Python + un menu simple
- Personne 4 : c'est un script qui appelle les fonctions existantes avec des `time.sleep()`

Tout est faisable en **2 heures maximum** par personne, même sans grande expérience Python.



Checklist finale pour chaque personne

- ☐ J'ai cloné le repo GitHub
- ☐ J'ai configuré mon `user.name` et `user.email` Git
- ☐ J'ai créé ma branche `feature/...`
- ☐ J'ai codé ma fonctionnalité dans les fichiers indiqués
- ☐ J'ai ajouté au moins 2 tests pytest pour ma fonctionnalité
- ☐ J'ai lancé `pytest tests/` et tout passe encore
- ☐ J'ai fait mon commit sous mon vrai nom
- ☐ J'ai poussé ma branche sur GitHub
- ☐ J'ai créé une Pull Request
- ☐ J'ai aussi terminé ma partie non-code (Figma / rapport / slides...)

Bon code à toute l'équipe.